

Практическая работа 6 по Информационным технологиям

Подпрограммы

Подпрограмма — это самый обычный программный код, находящийся в памяти там, куда его поместил транслятор. Выполняться она должна *только* по инструкциям **jal** или **jalr**, непосредственный переход на подпрограмму смысла не имеет. В частности, надо предпринять специальные меры, чтобы счётчик команд не дошагал до подпрограммы естественным путём (в примере используется специальный системный вызов «завершить программу»).

1. Прозрачность требует
 1. отдельной конвенции (о протоколе передачи параметров и возвращаемых значений);
 2. механизма сокрытия локальных меток.
2. Проблема **вложенного вызова** возникает, когда подпрограмма вызывается *из другой подпрограммы*:
 1. текущий адрес возврата надо сохранять перед вложенным вызовом и восстанавливать перед возвратом;
 2. конвенция должна предусматривать *цепочку* вложенных вызовов:
⇒ регистров на всю цепочку не хватит, их тоже надо где-то сохранять/восстанавливать.
3. Проблема **рекурсивного вызова** возникает, когда в цепочке вызовов некоторая подпрограмма встречается более одного раза (т. е. в конечном счёте вызывает сама себя); это не такая редкая ситуация: например, один вызов подпрограммы «разместить в памяти структуру данных» может для сложной структуры привести к нескольким вызовам той же подпрограммы для составных частей этой структуры.
4. Локальные данные могут быть изменены во вложенном вызове, поэтому их надо где-то *динамически* заводить/сохранять/освобождать.

Динамически выделять память удобнее всего на стеке. В начале подпрограммы все регистры, значения которых согласно конвенции следует сохранить до выхода из подпрограммы, а также регистр возврата **ra** записываются в стек операцией **push**. Перед выходом эти значения снимаются со стека (в обратном порядке) операцией **pop**. Эти значения, как правило, не используются внутри подпрограммы, важна только последовательность сохранения и восстановления.

В самом простом случае сохранять надо только **ra**, как показано в этом примере:

```
#          ...код программы
jal      subr1
#          ...код программы
#          ...конец программы
subr1:    addi    sp sp -4          # сохраним текущий ra
          sw      ra 0(sp)         # на стеке
#          ...код подпрограммы 1
jal      subr2                    # вызов изменяет значение ra
#          ...код подпрограммы 1
          lw ra 0(sp)              # восстановим текущий ra
```

```

        addi    sp sp 4          # из стека
        ret
subr2:   addi    sp sp -4        # сохраним ra
        sw      ra 0(sp)        # на стеке
#
        ...код подпрограммы 2
        lw      ra 0(sp)        # восстановим ra
        addi    sp sp 4          # из стека
        ret

```

Подпрограмма **subr2** — конечная, и в ней не обязательно сохранять и восстанавливать регистры. Но с точки зрения дисциплины программирования удобнее *все* подпрограммы оформлять одинаково. Возможно, в процессе работы над **subr2** из неё понадобится вызвать ещё подпрограмму, и она перестанет быть конечной. По всей видимости, надо исключить *любые* попытки подпрограмм сохранять какие-то данные не на стеке, т. к. в случае рекурсивного вызова не только **ra** и сохраняемые регистры.

Рассмотрим пример подпрограммы, вычисляющей функцию факториала. **Вопреки здравому смыслу** напишем эту подпрограмму рекурсивно: $f(n)! = f(n-1) * n$. Возникает одно значение — n — которое должно «переживать» рекурсивный вызов. Воспользуемся конвенцией, гарантирующей нам неизменность регистров s^* , и запишем n в регистр $s1$. Тогда для выполнения условий конвенции текущее значение этого регистра надо сохранять в стеке в начале подпрограммы, и восстанавливать перед возвратом из неё.

```

        jal     fact            # Параметр уже в a0
        li      a7 1
        ecall
        li      a7 10
        ecall
fact:   addi    sp sp -8        ## Запасаем две ячейки в стеке
        sw      ra 4(sp)        ## Сохраняем ra
        sw      s1 0(sp)        ## Сохраняем s1
        mv      s1 a0           # Запоминаем N в s1
        addi    a0 s1 -1        # Формируем n-1 в a0
        li      t0 1
        ble     a0 t0 done      # Если n<2, готово
        jal     fact            # посчитаем (n-1)!
        mul     s1 s1 a0        # s1 пережил вызов
done:   mv      a0 s1           # Возвращаемое значение
        lw      s1 0(sp)        ## Восстанавливаем sp
        lw      ra 4(sp)        ## Восстанавливаем ra
        addi    sp sp 8         ## Восстанавливаем вершину стека
        ret

```

В предыдущем примере мы формально воспользовались конвенцией о сохранении регистров. Сохранив регистр $s1$, мы обеспечили себе право использовать его как динамическую переменную.

Пролог и эпилог — начальная и завершающая части подпрограммы, которые обслуживают соблюдение конвенции, а к решаемой задаче имеют только косвенное отношение. Так, пролог в примере сохраняет на стеке два регистра, а использовалось бы их

больше — сохранял бы больше; эпилог эти два регистра (s0 и ra) восстанавливает (отмечены двойным знаком комментария ##). Формирование возвращаемого значения в литературе традиционно считается частью эпилога, т. к. её нельзя пропускать.

Похожий код получился бы без использования сохраняемого регистра s*. Пролог и эпилог уменьшались, но накапливающееся произведение всё равно надо записать (на стеке), и делать это придётся непосредственно перед вызовом подпрограммы, которая может испортить не сохраняемые регистры.

```
        jal    fact          # Параметр уже в a0
        li     a7 1
        ecall
        li     a7 10
        ecall
fact:    addi    sp sp -4
        sw     ra 0(sp)      # Сохраняем ra
        mv     t1 a0         # Запоминаем N в t1
        addi   a0 t1 -1      # Формируем n-1 в a0
        li     t0 1
        ble    a0 t0 done    # Если n<2, готово
        addi   sp sp -4      ## Сохраняем t1 на стеке
        sw     t1 0(sp)      ##
        jal    fact          # Посчитаем (n-1)!
        lw     t1 0(sp)      ## Вспоминаем t1
        addi   sp sp 4        ##
        mul    t1 t1 a0       # Домножаем на (n-1)!
done:    mv     a0 t1         # Возвращаемое значение
        lw     ra 0(sp)      # Восстанавливаем ra
        addi   sp sp 4        # Восстанавливаем вершину стека
        ret
```

Подготовка к вызову подпрограммы, которую делает вызывающая программа, называется преамбулой, а действия после возврата из подпрограммы почему-то не называются никак, так что назовём их финалом. В примере преамбула и финал обозначены двойным символом комментария ##.

Отличие конструкции «пролог-эпилог» от «преамбула-финал» — в области ответственности. За исключением того, что *явно сказано в конвенции*, вызываемая подпрограмма ничего не знает о том, что происходит в преамбуле и финале, а вызывающая программа — о том, что происходит в прологе и эпилоге. Например, не нужно знать, сколько и каких регистров сохраняется на стеке в прологе данной подпрограммы, раз уж конвенция гарантирует нам сохранность регистров типа s*.

В процессе планирования вычислений на языке ассемблера постоянно происходит так, что значение, соответствующее некоторому объекту программы (например, текущее посчитанное значение факториала в примерах), в разное время представлено различными аппаратными средствами: это может быть ячейка памяти, которую затем загрузят в регистр, регистр этот могут положить на стек — и всё это время с точки зрения алгоритма это будет один и тот же объект.

Такое отношение к данным резко отличается от более высокоуровневого понятия «переменная», в котором предполагается, что представление объекта и способ работы с ним всегда одинаковы. Можно сказать, что в языке ассемблера *нет переменных*, а есть только метки, и это не одно и то же.

В тексте мы будем использовать понятие *локальная переменная* преимущественно для такого случая:

- Предположим, что в нашей подпрограмме используется так много объектов, что для всех них не хватает регистров, или мы по каким-то другим причинам хотим хранить некоторое значение не в регистре, а в памяти.
- Понятно, что это значение надо хранить на стеке. И для того, чтобы не путаться, куда в данный момент указывает **sp**, выделение и инициализацию такой памяти на стеке удобно совмещать с прологом, а освобождение — с эпилогом.

```
subr:    addi    sp sp -12      ## Выделяем три ячейки — под ra и
                                     ## две переменные
        sw     ra 8(sp)        ## Сохраняем ra
        sw     zero 4(sp)      ## Инициализируем первую переменную нулём
        sw     zero 0(sp)      ## Инициализируем вторую переменную нулём
        # ...
        lw     t0 4(sp)        # Загружаем 1-ю локальную переменную в t0
        # ...
        sw     t1 0(sp)        # Записываем t1 во 2-ю локальную переменную
        # ...
        lw     ra 8(sp)        # Восстанавливаем ra
        addi    sp sp 12       # Освобождаем три ячейки стека
        ret
```

- Можно использовать сколько угодно локальных переменных.
- Переменные рекомендуется *инициализировать*:
 - В этом месте стека наверняка лежит какой-нибудь мусор, оставшийся от предыдущих вызовов подпрограмм.
 - Популярная ошибка — не проинициализировать переменную, а потом решить, что в ней лежит 0 (как это часто, *но не всегда* бывает).
 - Если не изменять стек в процессе работы, у переменных всегда будут фиксированные смещения относительно вершины стека.

Простая конвенция для универсальных подпрограмм

К конвенции для концевых вызовов п/п (подпрограммы) необходимо добавить пролог и эпилог, а также определить возможность преамбулы. Как обычно, конвенция не описывает нечто незыблемое, а предлагает некоторую дисциплину программирования, цель которой — удобство и эффективность.

1. Передаваемые п/п значения надо заносить в регистры **a***.
2. Вызов п/п должен производиться командой **jal** или **jalr**.
3. Никакая вызываемая п/п не модифицирует содержимое стека выше того указателя, который она получила в момент вызова.
4. П/п обязана сохранить на стеке значение **ra**.

5. П/п обязана сохранить на стеке все *используемые ею* регистры **s***.
6. П/п *может* хранить на стеке произвольное количество переменных. Количество этих переменных и занимаемого ими места на стеке не оговаривается и *может меняться* в процессе работы п/п.
7. Возвращаемое п/п значение надо заносить в регистры **a0, a1**.
8. П/п должна освободить все занятые локальными переменными ячейки стека.
9. П/п обязана восстановить из стека сохранённые значения **s*** и **ra**.
10. П/п обязана при возвращении восстановить значение **sp** в исходное.
11. Возврат из подпрограммы должен производиться командой **ret**.

Некоторые требования конвенции выглядят неоправданно строгими, например, чёткое предписание, где хранить переменные и регистры. Однако такая строгость резко упрощает повторное использование и отладку программ: даже не читая исходный текст программист точно знает, где искать адрес возврата, сохранённые регистры и локальные переменные. Конвенции с хранением данных на стеке крайне чувствительны к нарушению его цельности. Стоит «промахнуться» на ячейку (например, забыть про атомарность процедуры снятия со стека и не увеличить **sp**), и инструкции эпилога сохраняют все значения не по своим местам: адрес возврата останется на стеке, в регистр **ra** попадёт что-то из сохраняемого регистра, сохраняемые регистры получат «чужое» содержимое и т. д. При попытке выполнить переход на такой **ra** в лучшем случае возникнет исключение перехода в запрещённую память (хорошо, что малые числа соответствуют зарезервированным адресам и переход на адреса в секции данных запрещён). В худшем случае содержимое **ra** случайно окажется из допустимого диапазона секции кода, произойдёт возврат по этому адресу и начнётся выполнение лежащего там кода. Отладка такой ситуации (называемой «stack smash», снос стека) — непростая задача для программиста.

Для выполнения заданий вам могут потребоваться команды целочисленного деления:

div rd, rs1, rs2 # **rs1** – делимое, **rs2** – делитель, **rd** – целочисленное частное

и получения остатка после целочисленного деления:

rem rd, rs1, rs2 # **rs1** – делимое, **rs2** – делитель, **rd** – остаток от целочисленного деления

Задание 1 для нечётных номеров студ. билетов

Напишите программу, которая читает как константы из памяти **N** натуральных (возможно, отрицательных) целых чисел, затем выводит (в строку) **N** самых левых десятичных цифр этих чисел. Для вычисления одной цифры напишите подпрограмму **Left**, которая в **a0** получает число, и там же возвращает цифру. Формула для вычисления **N**, где **m** – это самая младшая цифра студ. билета:

если $m < 4$, то $N = m + 3$;

если $m > 7$, то $N = m - 3$.

Случайные натуральные числа со знаком придумайте сами.

Например, для последовательности 7 чисел:

-385, 23344, 8556, 9, -4, 677, 7889

Программа должна напечатать:

3289467

Задание 2 для чётных номеров студ. билетов

Напишите полную программу, которая читает как константы из памяти два натуральных числа $n1$ и $n2$ и выводит их наибольший общий делитель (НОД). Для вычисления НОД напишите рекурсивную подпрограмму *nod*, которая принимает два числа в **a0** и **a1** и возвращает значение в **a0**. Используйте последние 3 цифры вашего номера студенческого билета как число x . Например, для номера 431634: $x = 634$. Числа n зависят от x : $n1 = x$, $n2 = x // 8 + 43$.

Подсказка: можно использовать известный алгоритм Евклида, использующий остаток от деления:

1. $\text{НОД}(a, b) \{$
2. если a делится на b без остатка то - верни b ;
3. если b делится на a без остатка то - верни a ;
- 4.
5. если $a > b$ - то верни $\text{НОД}(a \bmod b, b)$;
6. иначе верни $\text{НОД}(a, b \bmod a)$;
7. $\}$

Тут *mod* — операция получения остатка от деления.

Контрольные вопросы:

1. Какой командой надо осуществлять вызов подпрограммы?
2. Что должна сделать подпрограмма при возвращении со значением указателя стека?
3. Какие регистры надо сохранять при вызове подпрограммы?